

BigTech Electronics Store

Retail journey, Java order coordination, and Ezzie support

Project type	Full stack ecommerce demonstration
Catalogue	28 products across 7 categories
Backend	Java and Spring Boot
Prepared by	Harika

1. Executive summary

BigTech is a responsive Indian electronics store demonstration with 28 products across phones, laptops, televisions, audio, appliances, wearables, and gaming.

The complete customer journey covers discovery, stock, cart, checkout, successful and failed payments, confirmation, delivery estimate, order history, account, and Ezzie support.

A Spring Boot service implements the order sequence independently from the browser demo and is verified with JUnit.

2. Problem and purpose

A product grid alone does not demonstrate ecommerce behaviour. Cart quantities, stock changes, payment outcomes, delivery estimates, order history, and support must agree.

The project purpose is to demonstrate those connected rules at final-year student scope while keeping every commercial integration simulated.

Success means both positive and failure paths work, tests cover the core rules, and public documentation states the boundaries clearly.

3. Customer journey

Customers search, filter, sort, inspect stock, and add an available product. Cart quantities cannot exceed the published demo stock.

Checkout collects a demonstration address and a selected payment outcome without requesting card, bank, or UPI details. Stock is checked again before payment.

Confirmed orders receive an order number and estimated date. Failed payments stay visible without a fulfilment promise. The account area links saved address, orders, and Ezzie.

Step	Outcome
Discover	Search, category, sort, price, rating, and stock
Cart	Quantity control and INR totals
Checkout	Address validation and payment simulation
Confirmation	Order number and delivery estimate
Account	Saved profile, address, orders, and support

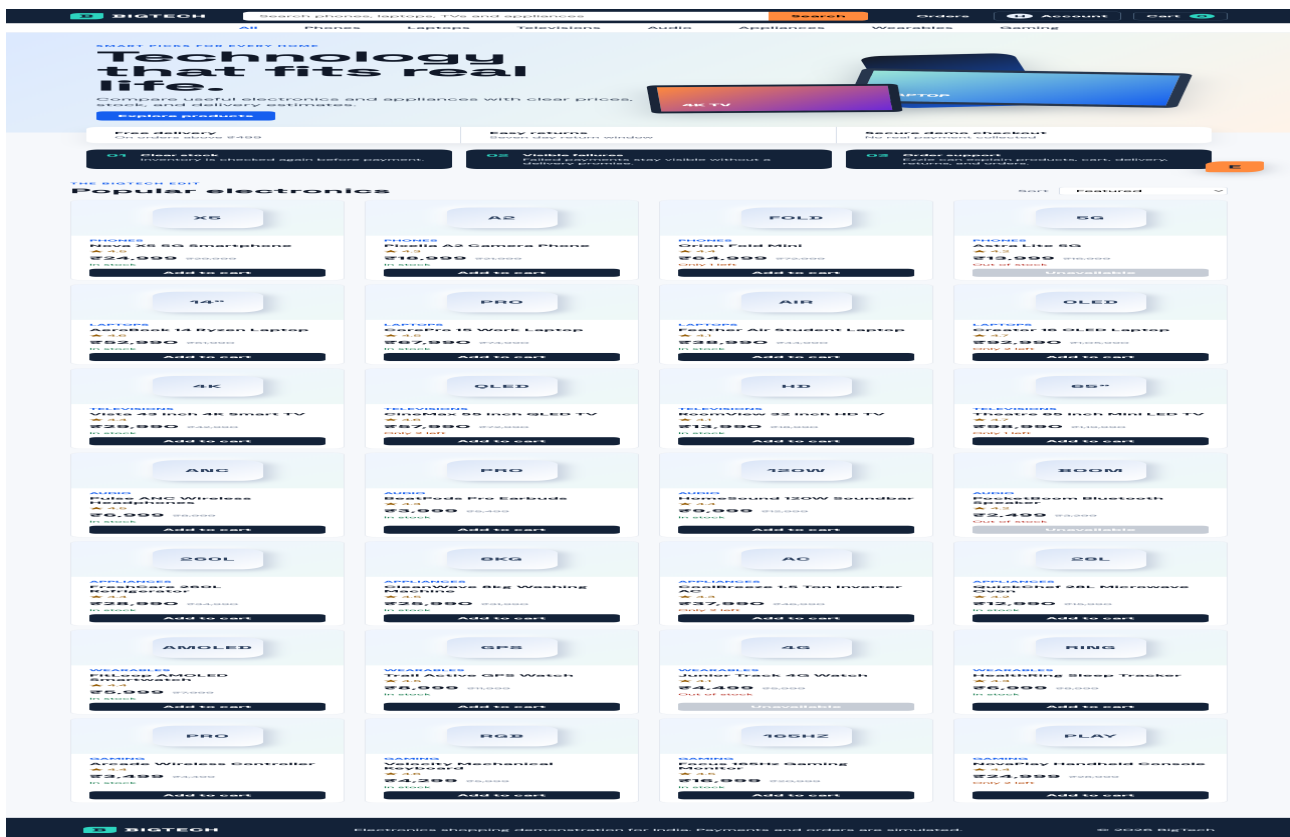
4. Storefront design

The interface uses a dark navy retail header, focused search, category navigation, a light technology hero, service promises, responsive product cards, and clear price and stock states.

The information architecture keeps orders, account, and cart available in the header. On mobile, the grid, checkout, and account sections collapse to one column.

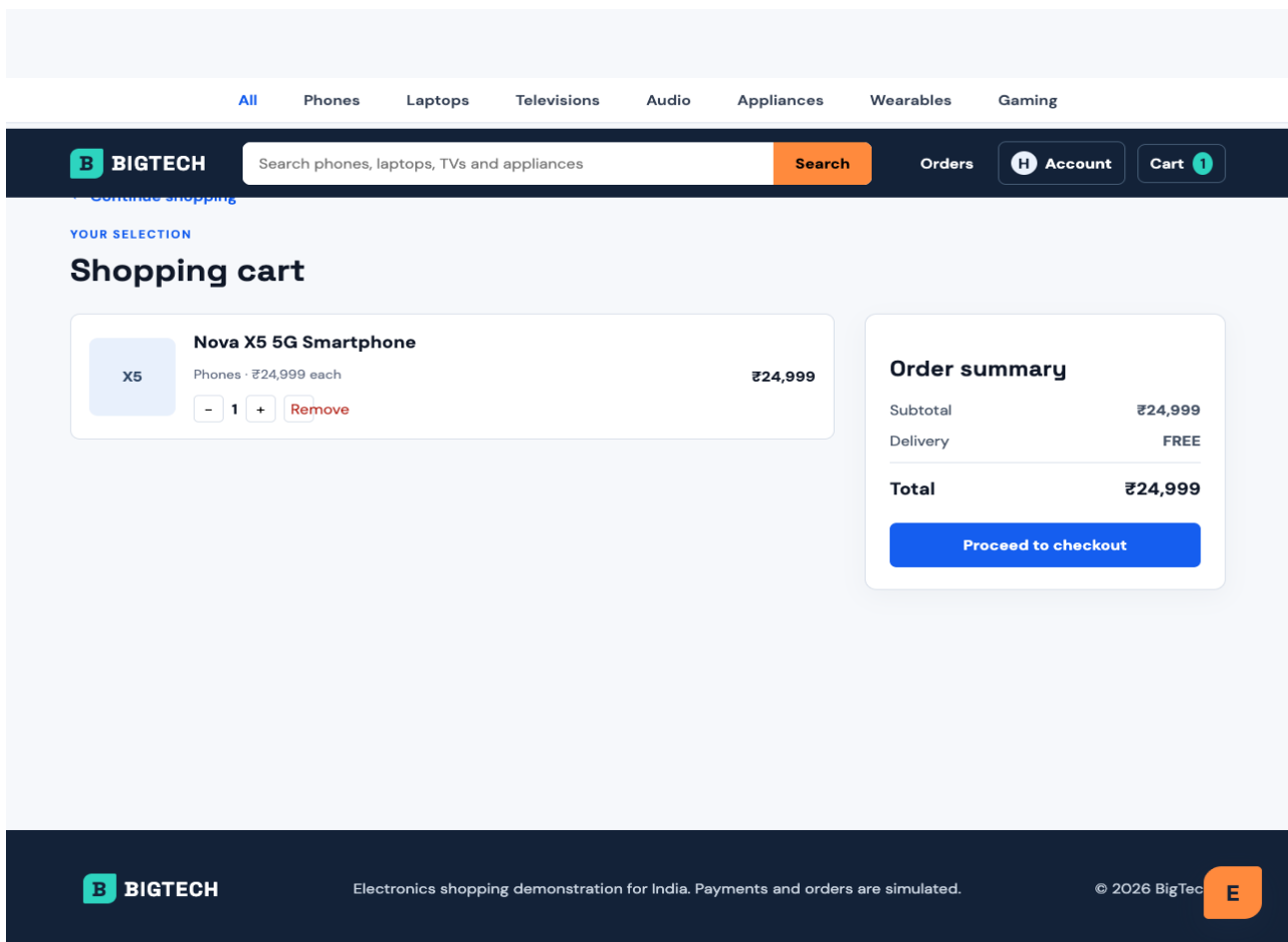
The deployed site is intentionally lightweight and does not require an account or external API to demonstrate the journey.

5. Verified storefront



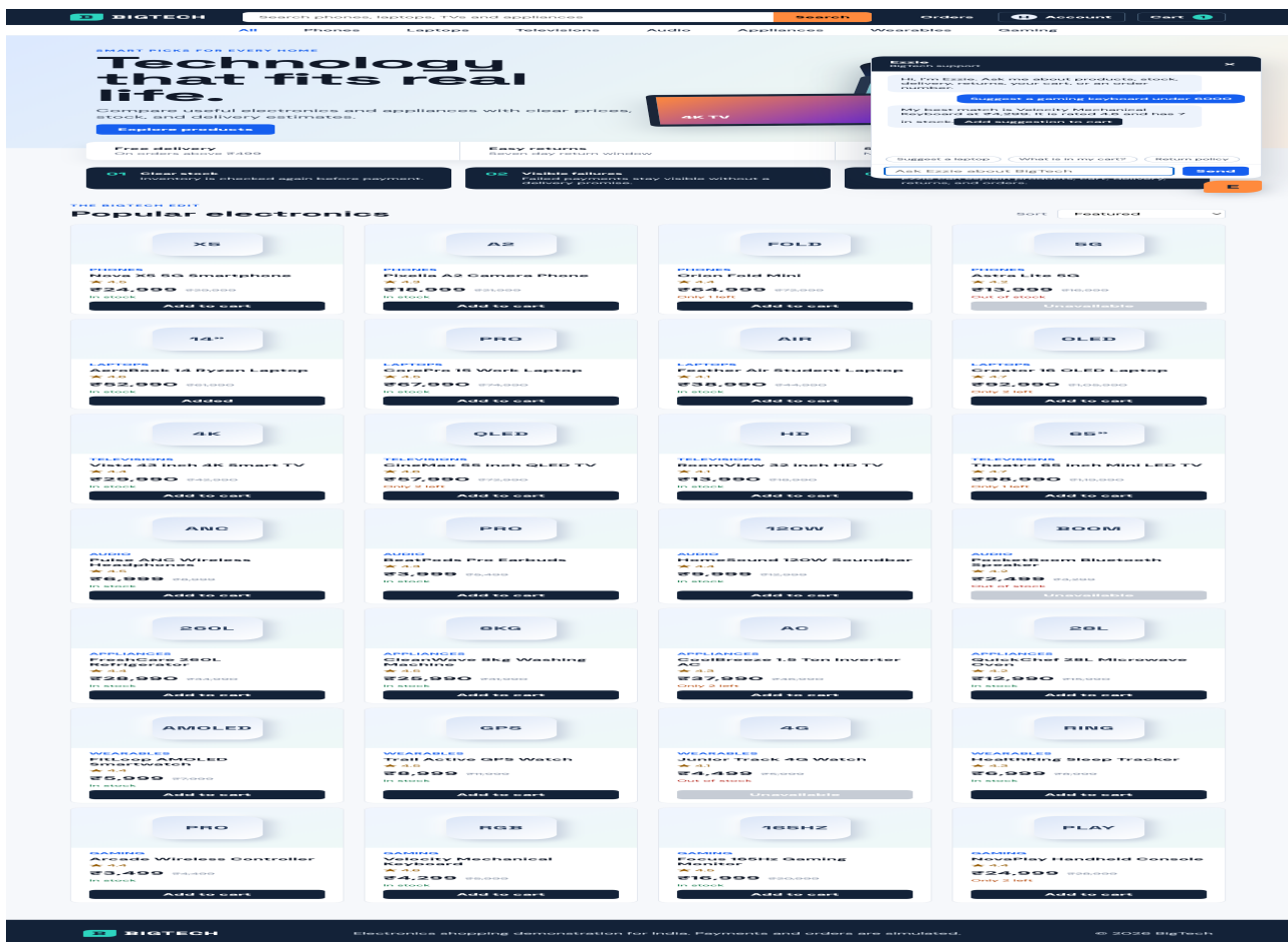
The storefront evidence is captured from the running application and shows the same catalogue and stock rules used by browser tests.

6. Cart and checkout



The cart repeats product, quantity, price, delivery, and total. Checkout labels payment as a simulation and revalidates inventory before creating an order.

7. Ezzie support



Ezzie supports product discovery, budget, stock, cart, checkout, payment, delivery, returns, and saved order numbers. It stays within the website and can add a recommendation directly to the cart.

8. Java order coordination

The API validates order items, customer name, pincode, and payment outcome. Inventory runs before payment, and fulfilment runs only after authorisation.

Explicit failure codes distinguish STOCK_CHANGED from PAYMENT_FAILED. Large-item categories use a longer estimate and a Clock dependency keeps tests deterministic.

The controller, coordinator, inventory, payment, and fulfilment classes keep responsibilities small enough to explain and test.

Component	Responsibility
OrderController	Validate and return HTTP results
InventoryService	Check requested quantity
PaymentService	Simulate authorisation
FulfilmentService	Calculate an estimate
OrderCoordinator	Apply the sequence and failure rules

9. Verification, deployment, and limitations

Seven frontend tests cover stock, totals, failed payment, Ezzie scope, budget recommendation, catalogue coverage, and gaming intent. Three Java tests cover confirmation, stock rejection, and failed payment.

Browser verification completes a successful order, confirms history, and checks a failed-payment path. The public site is deployed on Vercel with clean URLs and a single-page rewrite.

The project has no real authentication, payment gateway, shared database, courier integration, or live inventory. Browser state is device-local and all customer and order data is simulated.

Scenario	Expected behaviour
Out of stock	Cannot add
Changed stock	Checkout stops before payment
Failed payment	No delivery date
Successful payment	Order number and estimate
Outside Ezzie scope	Website-only boundary